SCIKEY DATABASE — Tuning & Design Document

Oracle 19c | PDB: pdb | Schema: SCIKEY Last Updated: April 2026

Table of Contents

SCIKEY DATABASE — Tuning & Design Document **1**
- Table of Contents 1
- 1. Design Philosophy 2
- 2. Tablespace Architecture 2
  - Definitions 2
  - Justification 2
- 3. Partitioning Strategy 3
  - 3.1 KEYWORD — RANGE Partitioning on keyword_text 3
  - 3.2 DOCUMENT — Composite LIST × HASH Partitioning 4
  - 3.3 DOC_KEYWORD, DOC_AUTHOR, AUTHOR_ORGANISM — Reference Partitioning 5
- 4. Base Table Index Design 6
  - 4.1 KEYWORD — Local B-tree Indexes 6
  - 4.2 DOCUMENT — Local B-tree and Bitmap Indexes 6
  - 4.3 DOC_KEYWORD — Local B-tree Indexes 7
  - 4.4 DOC_AUTHOR — Local B-tree and Bitmap Indexes 7
  - 4.5 AUTHOR_ORGANISM — Local B-tree Composite Index 8
  - 4.6 AUTHOR — Global B-tree Indexes 8
- 5. Materialized View Design 9
  - 5.1 MV_DOC_AUTHOR_KEYWORD 9
  - 5.2 MV_AUTHOR_DOC_COUNTS 10
  - 5.3 MV_AUTHOR_ORGANISM_DISTINCT 10
- 6. Materialized View Index Design 11
  - MV_DOC_AUTHOR_KEYWORD Indexes 11
  - MV_AUTHOR_DOC_COUNTS Indexes 11
  - MV_AUTHOR_ORGANISM_DISTINCT Indexes 12
- 7. LOB Storage Design 12
- 8. Sequence Design 13
- 9. Design Decisions Not Taken 13

---

1. Design Philosophy

SCIKEY is a read-heavy analytical database built on top of the HAL open archive. Its workload is dominated by three patterns: keyword prefix searches across a large and skewed text column, author name lookups joined to document metadata, and aggregations over author-organism affiliations. Write activity occurs in bulk during periodic HAL ingestion loads rather than as continuous OLTP inserts.

These characteristics drive every design choice in the schema:

- **Partitioning is used aggressively** to enable partition pruning on the most selective filter columns, reducing I/O to a fraction of the total data for common query patterns.
- **Materialized views pre-join the star schema** so that frequent multi-table queries never touch the base tables at all.
- **Local indexes are preferred** on partitioned tables so that index maintenance and pruning align with partition operations.
- **Bitmap indexes are used selectively** on genuinely low-cardinality columns where B-tree indexes would be wasteful.
- **Tablespaces are separated by data type** (table data vs index segments) to allow independent sizing, I/O tuning, and backup policy.

---

2. Tablespace Architecture

Definitions

```
CREATE TABLESPACE scikey_data
   DATAFILE '/u01/app/oracle/oradata/SCIKEY/scikey_data01.dbf'
   SIZE 10G AUTOEXTEND ON NEXT 1G MAXSIZE 25G;

CREATE TABLESPACE scikey_index
   DATAFILE '/u01/app/oracle/oradata/SCIKEY/scikey_index01.dbf'
   SIZE 2G AUTOEXTEND ON MAXSIZE 5G;
```

Justification

Two tablespaces, one for data and one for indexes, is the standard Oracle separation for analytical databases. The primary reasons are:

I/O profile separation. Table segment I/O is largely sequential during full scans and bulk loads. Index segment I/O is random during B-tree lookups. Placing them in separate tablespaces allows the DBA to assign them to different storage tiers (e.g., NVMe for indexes, SAS for bulk data) without reorganising the schema.

Independent sizing. `scikey_data` is provisioned at 10G with 1G autoextend increments up to 25G, reflecting that it holds all table data, LOB segments, and materialized view data — the large and growing portions of the schema. `scikey_index` is provisioned smaller (2G, max 5G) because index segments are a fraction of the data they cover, and because the autoextend ceiling enforces a deliberate constraint: if the index tablespace fills unexpectedly, it signals either a runaway index rebuild or missing segment maintenance rather than silently consuming disk.

Backup and recovery flexibility. Separating tablespaces allows RMAN to back up data and index segments on different schedules. Index segments can in principle be recovered by rebuilding rather than restoring from backup, so a less frequent backup schedule for `scikey_index` is defensible.

User quotas. The SCIKEY user is granted `QUOTA UNLIMITED` on both tablespaces explicitly, rather than relying on the RESOURCE role's default unlimited quota, ensuring that quota enforcement is intentional and visible in the DDL.

---

3. Partitioning Strategy

3.1 KEYWORD — RANGE Partitioning on keyword_text

```
PARTITION BY RANGE (keyword_text)
(
   PARTITION p_kw_a  VALUES LESS THAN ('B'),  -- 61,393 rows
   PARTITION p_kw_c  VALUES LESS THAN ('D'),  -- 87,884 rows
   PARTITION p_kw_s  VALUES LESS THAN ('T'),  -- 81,533 rows
   ...
   PARTITION p_kw_other VALUES LESS THAN (MAXVALUE)
)
```

Why RANGE on keyword_text?

Keyword lookup is the most frequent query pattern in the system. Every keyword search uses a fixed-prefix LIKE predicate (e.g., `keyword_text LIKE 'algorithm%'`). RANGE partitioning on the text column means Oracle can evaluate the partition boundary at parse time and restrict access to a single partition — this is confirmed in the Q10 execution plan where `Pstart=25, Pstop=25` shows only one of 25 partitions is accessed.

Without partitioning, a keyword prefix scan would scan the entire 868K-row table. With RANGE partitioning, it scans at most one partition. For the largest partitions (C at 87,884 rows, S at 81,533 rows) this is still meaningful work, but it is bounded and predictable.

Why individual letter partitions rather than grouping?

The distribution is skewed. C holds 87,884 rows, P holds 77,618 rows, and M holds 68,914 rows, while J holds only 6,230 and K only 5,680. Grouping letters into ranges would either create unbalanced partitions or require complex boundary definitions. Individual letter partitions keep each partition's size proportional to the natural distribution of the HAL keyword vocabulary, and make it straightforward to add subpartitioning to a single heavy letter in the future without restructuring the entire table.

Why p_kw_xyz groups X, Y, Z together?

X (1,591), Y (1,085), and Z (2,124) are small enough that individual partitions would add administrative overhead with no practical benefit. They are grouped under a single partition with an upper boundary of `'a'` (lowercase), which also has the effect of catching any uppercase-only entries that fall lexicographically between 'X' and 'a' in Oracle's default byte-order collation.

Why p_kw_other catches numerics and Cyrillics?

The 20,035 non-Latin entries (numerics, Cyrillics, other scripts) cannot be mapped to letter-based RANGE boundaries. The MAXVALUE catch-all partition is the standard Oracle pattern for this. These entries are a small fraction of the total (2.3%) and are expected to be queried less frequently in a Latin-script scientific corpus.

3.2 DOCUMENT — Composite LIST × HASH Partitioning

```
PARTITION BY LIST (discipline)
SUBPARTITION BY HASH (hal_document_id)
(
   PARTITION p_unknown   VALUES (NULL)                  SUBPARTITIONS 16,
   PARTITION p_cs        VALUES ('Computer Science')    SUBPARTITIONS 8,
   PARTITION p_chemeng   VALUES ('Chemical Engineering') SUBPARTITIONS 4,
   PARTITION p_marketing VALUES ('Marketing')           SUBPARTITIONS 2,
   PARTITION p_polisci   VALUES ('Political Science')   SUBPARTITIONS 2,
   PARTITION p_civil     VALUES ('Civil Engineering')   SUBPARTITIONS 2
```

)

Why LIST on discipline?

`discipline` is a low-cardinality column with a known, fixed set of values corresponding to HAL's top-level classification scheme. LIST partitioning is the natural choice when the partition key is categorical rather than ordered — RANGE would require artificial boundary definitions for text categories. When queries filter by discipline (e.g., `WHERE discipline = 'Computer Science'`), Oracle can prune to a single partition immediately.

Why HASH subpartitioning on hal_document_id?

The discipline distribution is heavily skewed: p_unknown alone holds 524,396 rows (about 70% of the table) because many HAL documents are not classified. A single LIST partition for NULL discipline would create one enormous segment. HASH subpartitioning on `hal_document_id` distributes rows within each discipline partition across multiple equal-sized subpartitions, which serves two purposes:

- It prevents any single segment from becoming a hot spot during parallel loads or concurrent queries against the same discipline.
- The subpartition count is scaled to row volume: p_unknown gets 16 subpartitions (524K rows ÷ 16 ≈ 33K per subpartition), p_cs gets 8 (133K ÷ 8 ≈ 17K), and the smaller disciplines get 2. This keeps subpartition sizes roughly comparable across the table.

`hal_document_id` is used as the HASH key rather than `document_key` because it is the natural business key from HAL and distributes more uniformly — sequence-generated surrogate keys (document_key) also distribute well, but using the natural key avoids any risk of sequential clustering if the sequence is cached and assigned in bursts.

Why NULL is its own LIST partition?

Oracle does not place NULLs in RANGE partitions by default, but in LIST partitioning NULL must be explicitly listed as a partition value or it will be rejected at insert time. Making NULL an explicit partition also signals design intent: unclassified documents are a known category in this dataset, not an exception.

3.3 DOC_KEYWORD, DOC_AUTHOR, AUTHOR_ORGANISM — Reference Partitioning

PARTITION BY REFERENCE (fk_dk_document)

Why reference partitioning?

These three tables are bridge tables whose entire purpose is to connect to `document`. Reference partitioning inherits the parent table's partition scheme through the foreign key, which means:

- A query that prunes `document` to partition `p_cs` will automatically also prune `doc_keyword`, `doc_author`, and `author_organism` to their corresponding partitions — no additional predicate is needed on the child tables.
- Partition maintenance operations (adding a partition, dropping a partition, moving a partition) cascade automatically from `document` to all reference-partitioned children. Without reference partitioning, each bridge table would need to be maintained independently and kept in sync.
- There is no partition key column duplication: the bridge tables do not need to store `discipline` or `hal_document_id` to benefit from pruning — the FK relationship carries the partition information.

The NOT DEFERRABLE constraint requirement. Oracle requires the foreign key used as a reference partition key to be `NOT DEFERRABLE`. This is a hard engine constraint: Oracle must be able to resolve the parent partition for every row at insert time, which requires the FK to be validated immediately. This constraint is already present in the DDL and is a non-negotiable design requirement for reference partitioning.

---

4. Base Table Index Design

4.1 KEYWORD — Local B-tree Indexes

```
CREATE INDEX idx_keyword_text       ON keyword(keyword_text) LOCAL;
CREATE INDEX idx_keyword_text_upper ON keyword(UPPER(keyword_text)) LOCAL;
```

idx_keyword_text (LOCAL): The primary access path for all keyword prefix lookups. LOCAL is mandatory here — a GLOBAL index on a RANGE-partitioned table would defeat partition pruning entirely, because a global index spans all partitions and Oracle would need to scan the full global index rather than the single-partition local index. With LOCAL, the index range scan stays within the same partition that pruning has already identified, as confirmed in Q10's plan (partition 25 only).

idx_keyword_text_upper (LOCAL, function-based): Supports case-insensitive searches where applications submit `UPPER(keyword_text) LIKE 'ALGORITHM%'`. Without this index, case-insensitive searches would require a full partition scan. The function-based index stores `UPPER(keyword_text)` as a virtual column, allowing Oracle to use an index range scan for these queries. It is LOCAL for the same pruning reasons as `idx_keyword_text`.

4.2 DOCUMENT — Local B-tree and Bitmap Indexes

```
CREATE INDEX idx_document_hal_id   ON document(hal_id_s) LOCAL;
CREATE INDEX idx_document_doi      ON document(doi_id) LOCAL;
CREATE BITMAP INDEX bidx_document_type       ON document(document_type) LOCAL;
CREATE BITMAP INDEX bidx_document_discipline ON document(discipline) LOCAL;
```

idx_document_hal_id and idx_document_doi (LOCAL B-tree): These support point lookups by the HAL string identifier and DOI — both are used in external integrations and citation resolution. They are LOCAL because `document` is composite-partitioned and queries arriving with only a `hal_id_s` or `doi_id` predicate (no discipline filter) will scan all partitions via the local index, which is still more efficient than a full table scan.

bidx_document_type and bidx_document_discipline (LOCAL Bitmap): `document_type` and `discipline` are low-cardinality columns — `document_type` has a small number of distinct values (article, conference paper, thesis, etc.) and `discipline` has exactly six distinct values plus NULL. Bitmap indexes are the correct choice for columns in this cardinality range because:

- A B-tree index on a column with 6 distinct values in 750K rows would have extremely poor selectivity and would rarely be chosen by the optimizer for single-column access.
- Bitmap indexes store one bit per row per distinct value, making them compact and efficient for `WHERE discipline = 'Computer Science'` filters and especially for multi-column bitmap AND/OR operations when combined with other bitmap predicates.
- They are LOCAL to align with the partition scheme and to avoid the write-lock contention that global bitmap indexes introduce during concurrent DML (bitmap indexes lock the entire bitmap segment during updates; local bitmaps limit lock scope to one partition).

4.3 DOC_KEYWORD — Local B-tree Indexes

```
CREATE INDEX idx_doc_keyword_document ON doc_keyword(document_key) LOCAL;
CREATE INDEX idx_dk_keyword           ON doc_keyword(keyword_key)   LOCAL;
```

`doc_keyword` is a pure bridge table with a composite primary key of `(document_key, keyword_key)`. The primary key index covers lookups starting from `document_key`, but lookups arriving from the keyword side (joining from `keyword` to `doc_keyword` as in Q10) need `keyword_key` as the leading column. `idx_dk_keyword` provides this reverse access path. Both are LOCAL because `doc_keyword` is reference-partitioned off `document` — local indexes respect and participate in the inherited partition pruning.

4.4 DOC_AUTHOR — Local B-tree and Bitmap Indexes

```
CREATE INDEX idx_doc_author_document ON doc_author(document_key) LOCAL;
```

```
CREATE INDEX idx_da_author          ON doc_author(author_key)   LOCAL;
CREATE BITMAP INDEX bidx_doc_author_quality ON doc_author(quality) LOCAL;
```

Same dual-direction access path logic as `doc_keyword`: `document_key` is covered by the primary key index; `idx_da_author` supports joins arriving from the author dimension. `bidx_doc_author_quality` is a bitmap index on the `quality` column, which encodes a small set of HAL-defined quality classifications (e.g., peer-reviewed, with referee, etc.). This is another genuine low-cardinality column where a bitmap is appropriate and a B-tree would be inefficient.

4.5 AUTHOR_ORGANISM — Local B-tree Composite Index

```
CREATE INDEX idx_ao_organism        ON author_organism(organism_key)          LOCAL;
CREATE INDEX idx_ao_document_author  ON author_organism(document_key, author_key)
LOCAL;
```

idx_ao_organism: Supports lookups from the organism dimension into the bridge table — the natural join direction when querying "which authors are affiliated with organism X".

idx_ao_document_author (composite): The primary key of `author_organism` is `(document_key, author_key, organism_key)`. The PK index covers access starting from `document_key`. However, because the FK to `doc_author` is on `(document_key, author_key)`, a query joining back to `doc_author` on both columns benefits from an index with exactly `(document_key, author_key)` as leading columns. The composite index provides this without including `organism_key`, which would make it redundant with the PK.

4.6 AUTHOR — Global B-tree Indexes

```
CREATE INDEX idx_author_name         ON author(last_name, first_name);
CREATE INDEX idx_author_lastname_upper ON author(UPPER(last_name));
```

`author` is an unpartitioned dimension table. There is no partitioning to align with, so these are global indexes by definition. LOCAL/GLOBAL distinction only applies to partitioned tables.

idx_author_name (composite, last_name leading): Author name lookups almost always filter on last name, with first name used as a secondary discriminator for common surnames. A composite index with `last_name` leading satisfies both `WHERE last_name = 'Smith'` and `WHERE last_name = 'Smith' AND first_name = 'John'` with a single range scan, and also supports ORDER BY on `(last_name, first_name)` without a sort operation.

idx_author_lastname_upper (function-based): Same rationale as `idx_keyword_text_upper` — supports `UPPER(last_name) LIKE 'SMITH%'` without a full table scan. Author names in HAL come from submitter input and are not normalised to a consistent case, making case-insensitive search a realistic requirement.

---

5. Materialized View Design

5.1 MV_DOC_AUTHOR_KEYWORD

```
CREATE MATERIALIZED VIEW mv_doc_author_keyword
BUILD IMMEDIATE REFRESH COMPLETE ON DEMAND AS
SELECT d.document_key, d.hal_document_id, d.hal_id_s, d.title,
       d.document_type, d.discipline, d.doi_id,
       da.author_key, da.quality,
       a.author_id_hal, a.first_name, a.last_name,
       dk.keyword_key, k.keyword_text, k.wikidata_id
FROM document d
JOIN doc_author da ON d.document_key = da.document_key
JOIN author a      ON da.author_key  = a.author_key
JOIN doc_keyword dk ON d.document_key = dk.document_key
JOIN keyword k     ON dk.keyword_key = k.keyword_key;
```

Purpose and justification. The most frequent query pattern in SCIKEY joins four tables: `document`, `doc_author`, `author`, `doc_keyword`, and `keyword`. Without this MV, every keyword or author search would execute this five-table join at query time against ~750K documents, ~868K keywords, and the bridge tables that connect them. At the data volumes in this schema, that join produces a Cartesian expansion — a document with 5 authors and 10 keywords generates 50 rows in the join result — making it expensive to compute on demand.

This MV pre-computes and stores the full join result. Q4 and Q7 both achieve cost 6 by accessing this MV through its keyword index rather than executing the underlying join. The cost of maintaining the MV (a full `REFRESH COMPLETE ON DEMAND` rebuild) is paid once per refresh cycle rather than on every query.

REFRESH COMPLETE ON DEMAND is chosen over FAST (incremental) refresh because:

- FAST refresh requires materialized view logs on all base tables, which adds DML overhead to every insert and update on `document`, `author`, `keyword`, `doc_author`, and `doc_keyword`. For a bulk-loaded database where writes are batched rather than continuous, this overhead is not justified.

- The join involves multiple tables with a many-to-many relationship (one document to many keywords, one document to many authors). Fast refresh of a multi-table MV with a many-to-many join is one of the most restrictive scenarios in Oracle — it requires specific query structure constraints and often silently falls back to complete refresh anyway.
- Complete refresh on a bulk-load schedule is predictable, simple to monitor, and restores the MV to a fully consistent state after each HAL ingestion.

What is excluded. `abstract`, `domain_codes`, and `url_primary` are intentionally excluded from this MV. These are CLOB columns and including them would increase the MV size dramatically, slow refresh, and pollute the buffer cache with LOB data during keyword and author searches that do not need abstract text. CLOB access is handled separately through Q10's direct base table path.

5.2 MV_AUTHOR_DOC_COUNTS

```
CREATE MATERIALIZED VIEW mv_author_doc_counts
BUILD IMMEDIATE REFRESH COMPLETE ON DEMAND AS
SELECT a.author_key, a.author_id_hal, a.first_name, a.last_name,
    COUNT(DISTINCT da.document_key) AS doc_cnt
FROM doc_author da
JOIN author a ON da.author_key = a.author_key
GROUP BY a.author_key, a.author_id_hal, a.first_name, a.last_name;
```

Purpose and justification. Q8 requests authors ranked by document count — a `GROUP BY` aggregation over the full `doc_author` bridge table. Computing this on demand would require scanning all rows of `doc_author` (which inherits the full document volume) and performing a `COUNT(DISTINCT)` aggregation every time. This MV pre-computes the count once and stores one row per author.

`COUNT(DISTINCT da.document_key)` is used rather than `COUNT(*)` because the bridge table can contain multiple rows per `(document_key, author_key)` pair if the same author appears in multiple keyword combinations for a document (though the primary key prevents this in `doc_author` specifically — using DISTINCT is a defensive measure against schema evolution).

idx_mv_adc_doc_cnt on `doc_cnt` is included to support `ORDER BY doc_cnt DESC` with a reverse index scan rather than a sort operation, which is the access pattern in Q8.

5.3 MV_AUTHOR_ORGANISM_DISTINCT

```
CREATE MATERIALIZED VIEW mv_author_organism_distinct
BUILD IMMEDIATE REFRESH COMPLETE ON DEMAND AS
SELECT DISTINCT
```

```
    a.author_key, a.author_id_hal, a.first_name, a.last_name,
    o.organism_key, o.struct_name, o.hal_structure_id
FROM author_organism ao
JOIN author a   ON ao.author_key   = a.author_key
JOIN organism o ON ao.organism_key = o.organism_key;
```

Purpose and justification. `author_organism` records affiliations at the per-document level — an author affiliated with the same institution across 50 papers generates 50 rows in the base table. Q9 and Q12 both need distinct `(author, organism)` pairs without caring which specific documents created the affiliation. The `SELECT DISTINCT` in this MV collapses all per-document duplicates at refresh time, so queries against it operate on a deduplicated set.

Without this MV, Q9's `COUNT(DISTINCT author_key)` per organism would need to deduplicate across the full `author_organism` base table on every execution — exactly the two-phase hash group-by shown in the Q9 execution plan, but against the larger base table rather than the already-distinct MV. The MV reduces the work to a single aggregation pass over a smaller, pre-deduplicated row set.

---

6. Materialized View Index Design

MV_DOC_AUTHOR_KEYWORD Indexes

```
CREATE INDEX idx_mv_dak_keyword_text ON mv_doc_author_keyword(keyword_text);
CREATE INDEX idx_mv_dak_last_name    ON mv_doc_author_keyword(last_name);
CREATE INDEX idx_mv_dak_dockey       ON mv_doc_author_keyword(document_key);
CREATE INDEX idx_mv_dak_authorkey    ON mv_doc_author_keyword(author_key);
```

idx_mv_dak_keyword_text is the most important index in the entire schema. It is the index used by Q4 and Q7 (confirmed by `IDX_MV_DAK_KEYWORD_TEXT` in both execution plans) and turns a potentially full-scan query into a cost-6 index range scan. Because the MV is not partitioned, this is a global B-tree index over the full MV row set — the single access path for all keyword prefix searches against the pre-joined data.

idx_mv_dak_last_name supports author name searches directly against the MV (Q5 in the stress test), avoiding any need to join back to the `author` base table for name-filtered queries.

idx_mv_dak_dockey and idx_mv_dak_authorkey support joins into the MV from other query patterns that arrive with a known `document_key` or `author_key` — for example, Q6's hash join uses `author_key` as the join column between the two MVs. The presence of

`idx_mv_dak_authorkey` is what enables the optimizer's adaptive plan note in Q6 to potentially switch to a nested loop with an index probe if the build side is small enough.

MV_AUTHOR_DOC_COUNTS Indexes

```
CREATE INDEX idx_mv_adc_last_name  ON mv_author_doc_counts(last_name);
CREATE INDEX idx_mv_adc_author_key ON mv_author_doc_counts(author_key);
CREATE INDEX idx_mv_adc_doc_cnt    ON mv_author_doc_counts(doc_cnt);
```

idx_mv_adc_last_name is the primary filter index for Q8, which applies a `last_name LIKE` predicate before sorting by `doc_cnt`.

idx_mv_adc_doc_cnt is a supporting index for the `ORDER BY doc_cnt DESC` in Q8. Whether Oracle uses it depends on selectivity — if the `last_name` filter is very selective, Oracle will filter first and sort the small result in memory. If the filter is broad, Oracle may use the `doc_cnt` index in reverse order to avoid a full sort. Having the index available gives the optimizer the choice.

MV_AUTHOR_ORGANISM_DISTINCT Indexes

```
CREATE INDEX idx_mv_aod_author_key   ON mv_author_organism_distinct(author_key);
CREATE INDEX idx_mv_aod_organism_key ON mv_author_organism_distinct(organism_key);
CREATE INDEX idx_mv_aod_last_name    ON mv_author_organism_distinct(last_name);
```

idx_mv_aod_author_key is the join index for Q6, which joins this MV to `MV_DOC_AUTHOR_KEYWORD` on `author_key`. As noted in the Q6 execution plan analysis, Oracle currently chooses a hash join with full scans of both MVs. This index exists to give the optimizer the option to switch to a nested loop if the `last_name` filter produces a small enough build side — the adaptive plan note in Q6 confirms Oracle is aware of this alternative.

idx_mv_aod_organism_key supports lookups by organism — queries of the form "which authors are associated with organism X" arrive with `organism_key` as the filter and benefit from a direct index probe rather than a full MV scan.

idx_mv_aod_last_name supports the `last_name LIKE` filter in Q6 and Q12. Currently this filter is applied as a post-scan predicate against a full MV scan (as seen in Q6's plan). Adding this index creates the possibility for Oracle to use a range scan on `last_name` to drive the query, which would be beneficial if the `last_name` filter is selective enough — for common surnames like 'S%' it likely is not, but for more specific prefixes it could reduce the 406K full scan materially.

---

7. LOB Storage Design

```
LOB (abstract, domain_codes, url_primary) STORE AS
   (TABLESPACE scikey_data ENABLE STORAGE IN ROW CHUNK 8192 NOCACHE)
```

ENABLE STORAGE IN ROW. LOBs smaller than approximately 4000 bytes are stored inline in the table row segment rather than in a separate LOB segment. For `domain_codes` and `url_primary`, which tend to be short strings, this avoids the double I/O of reading the row and then separately dereferencing the LOB pointer. For `abstract`, which varies widely, shorter abstracts benefit from inline storage while longer ones are stored out-of-row automatically.

CHUNK 8192. The chunk size defines the unit of LOB I/O — Oracle reads and writes LOB data in chunks of this size. 8192 bytes (8K) matches a common Oracle block size and is appropriate for `DBMS_LOB.SUBSTR` access patterns where a partial read of the first 200–500 characters of an abstract is the typical access. A larger chunk (e.g., 32K) would waste I/O for short reads; a smaller chunk would fragment large abstracts unnecessarily.

NOCACHE. LOB segments are excluded from the buffer cache by default. `NOCACHE` makes this explicit and deliberate. Abstract text is accessed infrequently and non-repetitively — caching it would evict more useful index and table data from the buffer pool. Queries that need abstract text (Q10) accept the direct read cost as appropriate for their usage frequency.

---

8. Sequence Design

```
CREATE SEQUENCE seq_keyword  START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_author   START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_organism START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_document START WITH 1 INCREMENT BY 1;
```

Four sequences generate surrogate primary keys for the four entity tables. They are defined without `CACHE` or `ORDER` clauses, meaning Oracle applies the default cache of 20 values per instance. For a bulk-loaded database this is adequate — the sequence contention that would affect a high-concurrency OLTP system is not a concern here.

The sequences start at 1 with increment 1, producing dense integer keys. This is intentional: dense keys produce compact B-tree index entries and sequential leaf block writes during bulk inserts, avoiding the index block contention that random or sparse keys can cause. The default columns use `DEFAULT seq_x.NEXTVAL` rather than a trigger, which is the preferred pattern in Oracle 12c and later as it avoids trigger overhead on every insert.

---

9. Design Decisions Not Taken

Understanding what was deliberately excluded clarifies the intent of the choices above.

No FAST refresh on materialized views. As discussed in Section 5, FAST refresh requires MV logs on all base tables and imposes continuous DML overhead. For a batch-loaded schema this is the wrong trade-off.

No global indexes on partitioned tables (except via PK/UK constraints). Global indexes span all partitions and are invalidated by partition DDL operations (adding, dropping, splitting partitions) unless `UPDATE INDEXES` is specified. For a schema where partition structure may evolve as new HAL disciplines are added, local indexes are safer and require no index rebuild after partition maintenance.

No composite indexes on base bridge tables beyond the primary key. `doc_keyword`, `doc_author`, and `author_organism` are accessed via their individual column indexes in combination — Oracle's index join or nested loop with multiple index scans handles these patterns well without maintaining additional composite segments. The one exception is `idx_ao_document_author`, which is composite because the FK to `doc_author` requires both columns and a composite index is more efficient than two separate single-column lookups for this specific join.

No indexes on CLOB columns. Oracle does not support standard B-tree indexes on CLOB columns. Full-text indexing via Oracle Text (`CTXSYS.CONTEXT`) would support keyword-within-abstract search, but this is out of scope for the current schema and would require a separate design decision covering index synchronisation, lexer configuration, and scoring.

No partitioning on AUTHOR or ORGANISM. These are small, stable dimension tables. AUTHOR is expected to have at most a few hundred thousand rows; ORGANISM is smaller still. The overhead of partition metadata, local index maintenance, and reference partition propagation is not justified for tables at this scale. Global indexes on these tables are simpler, faster to maintain, and sufficient.

SCIKEY Tuning & Design Document — update this document whenever partitioning structure, index definitions, MV definitions, or refresh schedules change.